

What is the COPY INTO command?

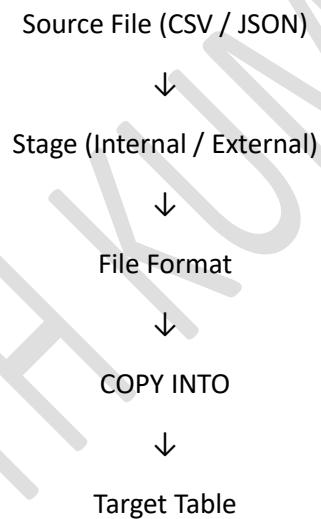
COPY INTO is used to **load data from files** (CSV, JSON, Parquet, Avro, ORC, XML)

- ➡ **from a stage**
- ➡ **into a Snowflake table.**

✦ In real-time projects:

- Source systems export data as **files**
- Files are stored in **S3 / Azure Blob / GCS**
- Snowflake **pulls data using COPY INTO**

COPY INTO Architecture (Flow)



Why COPY INTO is important (Real-Time Use)

Use Case	Explanation
Bulk data load	Load millions/billions of rows
Data lake ingestion	S3 → Snowflake
Daily / hourly batch loads	Via Tasks + Streams
Error handling	Control bad records
Audit & monitoring	Load history tracking

Types of COPY INTO

- ◆ **Type 1: Load data INTO a table (MOST USED)**

`COPY INTO table_name FROM @stage_name;`

- ◆ **Type 2: Unload data FROM table (export)**

`COPY INTO @stage_name FROM table_name;`

⚠ This session focuses on **loading INTO table**

Basic COPY INTO Syntax

`COPY INTO <target_table>`

`FROM <stage_location>`

`FILE_FORMAT = (<file_format_options>)`

`ON_ERROR = <error_option>;`

Step-by-Step Practical Example

✅ STEP 1: Create Database

`create database snowflake_exercise;`

`use database snowflake_exercise;`

Why we use this

- Database is a **logical container**
- Separates projects / applications

What happens internally

- Snowflake creates metadata for database
- No storage is consumed yet

Output

- Database object created
 - No tables, no data
-

✓ STEP 2: Create Schema

```
create schema sf_schema;
```

Why we use this

- Schema organizes objects (tables, stages, formats)
- Required before creating tables

Internally

- Schema metadata stored
- Acts as namespace

Output

- Empty schema created
-

✓ STEP 3: Create FILE FORMAT

```
create file format my_csv_format
```

```
type=csv
```

```
skip_header=1;
```

Why we use this

- Tells Snowflake how to read CSV
- Mandatory for structured file loading

Internally

- Parsing rules stored
- Used only during COPY execution

Output

- File format object created
-

Describe File Format

```
describe file format my_csv_format;
```

Why

- To verify parsing settings

Output

- Shows delimiter, skip header, type

✅ STEP 4: Create EMP Table

```
create or replace table emp (  
  empno number,  
  ename varchar(100),  
  job varchar(25),  
  mgr number,  
  hiredate varchar(100),  
  sal number,  
  comm number,  
  deptno number  
)
```

```
STAGE_FILE_FORMAT = (FORMAT_NAME = 'my_csv_format');
```

Why we use this

- Target table for COPY INTO
- STAGE_FILE_FORMAT sets **default file format**

Internally

- Table metadata created
- Micro-partition structure prepared

Output

- Empty table created
-

✅ STEP 5: GET_DDL (Object Script)

```
select get_ddl('table','emp');
```

Why we use this

- To **recreate objects**
- Used in migration & backup

Internally

- Reads metadata
- Generates creation SQL

Output

- DDL script text

✓ STEP 6: Create STAGE

External Stage

```
CREATE STAGE my_external_stg
```

```
URL = 's3://skillscaleraw/DW/'
```

```
CREDENTIALS = (
```

```
  AWS_KEY_ID='xxx'
```

```
  AWS_SECRET_KEY='yyy' );
```

Why we use this

- Stage = **bridge between Snowflake and S3**
- COPY reads files only via stages

Internally

- Stores S3 path + auth details
- No data movement yet

Output

- Stage object created
-

List Files in Stage

```
LIST @my_external_stg/employee/;
```

Why we use this

- Verify files exist
- Debug path issues

Output

- File names, sizes, timestamps
-

Attach File Format to Stage

```
ALTER STAGE my_external_stg
```

```
SET file_format = my_csv_format;
```

Why

- Avoid specifying file format every time

Output

- Stage updated

✅ STEP 7: Query Data Directly from Stage

```
select $1,$2,$3,$4,$5,$6,$7,$8  
from @my_external_stg/employee/emp.csv  
(file_format => 'my_csv_format');
```

Why we use this

- Preview data before loading
- Validate column order

Internally

- Reads file
- No data loaded into table

Output

- Virtual result set from file
-

✅ STEP 8: Basic COPY INTO

```
COPY INTO emp  
FROM @my_external_stg/employee/emp.csv  
file_format='my_csv_format';
```

Why

- Loads file data into table

Internally

1. Reads file
2. Applies file format
3. Maps columns
4. Loads rows

Output

- Rows inserted into EMP table
-

✅ STEP 9: Column Re-Ordering & Omission

```
COPY INTO emp (EMPNO,ENAME,JOB,MGR,HIREDATE,SAL)
FROM (
  select $1,$2,$3,$4,$5,$6
  from @my_external_stg/employee/emp.csv
)
(file_format=>'my_csv_format');
```

Why

- File column order $\neq$ table order
- Ignore unnecessary columns

Internally

- SELECT controls mapping
 - COPY inserts mapped columns only
-

✅ STEP 10: FORCE = TRUE

```
COPY INTO emp
FROM @my_external_stg/employee/emp.csv
FORCE=TRUE;
```

Why

- Reload same file again
- Used in testing

Internally

- Bypasses file load history

Output

- Duplicate rows inserted
-

✅ STEP 11: Load Specific Files

```
COPY INTO emp
FROM @my_external_stg/employee/
FILES=('emp.csv','emp_load1.csv');
```

Why

- Selective file loading

Internally

- Only listed files are read
-

✅ STEP 12: Pattern Based Loading

```
COPY INTO emp
FROM @my_external_stg/employee/
PATTERN='.*emp_load.*[0-9].*.csv';
```

Why

- Load files matching naming rule
 - Used in daily batch loads
-

✅ STEP 13: Data Type Casting

```
select $1, cast($4 as number),$2,$7,$8
from @my_external_stg/employee/emp.csv;
```

Why

- Fix data type mismatch
 - Avoid COPY failures
-

✅ STEP 14: TRUNCATECOLUMNS

```
COPY INTO emp
FROM @my_external_stg/employee/emp
TRUNCATECOLUMNS=TRUE;
```

Why

- Prevent string length errors

Internally

- Cuts extra characters
 - Loads remaining data
-

✅ STEP 15: COPY_HISTORY

```
select *  
from table(  
  information_schema.copy_history(  
    table_name=>'emp',  
    start_time=>dateadd(hours,-24,current_timestamp()) );
```

Why

- Audit & monitoring
- Debug failed loads

Output

- File name
 - Rows loaded
 - Errors
-

✅ STEP 16: ON_ERROR Handling

```
COPY INTO emp  
FROM @my_external_stg/employee/  
ON_ERROR=CONTINUE;
```

Why

- Load valid rows
- Skip bad rows

Options explained

- ABORT_STATEMENT → stop
- CONTINUE → skip rows
- SKIP_FILE → skip entire file
- SKIP_FILE_2 → skip if ≥2 errors
- SKIP_FILE_0.1% → skip if % exceeded

✅ STEP 17: VALIDATION_MODE (No Load)

```
COPY INTO emp
FROM @my_external_stg/employee/
VALIDATION_MODE=RETURN_ALL_ERRORS;
```

Why

- Validate data before loading

Internally

- Scans file
- No data inserted

Output

- Error rows only
-

✅ STEP 18: RESULT_SCAN (Store Errors)

```
select *
from table(result_scan(last_query_id()));
```

Why

- Capture validation errors

Internally

- Reads cached query result
-

✅ STEP 19: VALIDATE Table Function

```
select *
from table(validate(emp, job_id=>'_last'));
```

Why

- Capture rejected rows after COPY

Output

- Row data + error reason
-

✅ STEP 20: ACCOUNT_USAGE & HISTORY

```
select * from snowflake.account_usage.copy_history;
```

Why

- Long-term audit (365 days)

🎯 FINAL BIG PICTURE 🎯

Step	Purpose
Database & Schema	Organization
File Format	Parsing
Stage	Storage bridge
LIST	Validation
SELECT from Stage	Preview
COPY INTO	Load
ON_ERROR	Error control
VALIDATION_MODE	Pre-check
RESULT_SCAN	Error logging
COPY_HISTORY	Monitoring